YILDIZ TECHNICAL UNIVERSITY

FACULTY OF CHEMISTRY AND METALLURGY

DEPARTMENT OF MATHEMATICAL ENGINEERING

MATHEMATICAL ENGINEERING

SQL PROJECT

YTU ASSISTANT RECRUITMENT SYSTEM

Technical Report

Züleyha Uslu	**Akın Engin KILINÇ**	Kaan Gökkoca
21052071	22058005	22052100

Prepared by: Group 15
Course: MTM4692 — Database Systems

1. Problem Statement

Yildiz Technical University currently has no centralized system for managing assistant recruitment. Assistantship opportunities — such as teaching assistants, lab assistants, and research assistants — are shared in a scattered and informal way through emails, physical announcements, or word of mouth.

This creates two core problems: students find it difficult to track and discover available opportunities, and professors have no structured way to receive or manage applications. The process is opaque, inconsistent, and inefficient for both sides.

2. Why It Matters

Stakeholder	Current Problem	What the Platform Provides
Students	Miss opportunities due to scattered announcements; no way to track application status.	A single place to browse positions, apply, and follow application status.
Professors	Receive unorganized applications; no structured system to evaluate candidates.	Structured posting and applicant management tools.
University	No visibility into recruitment activity or outcomes.	A transparent, organized workflow within the institution.

3. Preliminary Schema Design

The database is hosted on Azure SQL and consists of 12 tables. All primary keys use IDENTITY(1,1). Cascade deletes are applied on all child tables to maintain referential integrity.

Table	Key Columns	Purpose
Users	`user_id PK, role, email UNIQUE, password_hash, first_name, last_name, faculty, department, is_verified`	Central user table for both roles.
Email_Verification	`id PK, user_id FK, verification_token, expires_at, verified_at`	Stores email confirmation tokens.
Student_Profile	`user_id FK/PK, degree_level, class_year, gpa, expected_graduation_date`	Extended profile data for students.
Professor_Profile	`user_id FK/PK, academic_title`	Extended profile data for professors.
Jobs	`job_id PK, user_id FK, job_category, department_name, position, description, deadline, is_active`	Assistantship postings created by professors.
Job_Applicants	`id PK, job_id FK, user_id FK, status, applied_at | UNIQUE(job_id, user_id)`	Student applications. Duplicate prevention enforced at DB level.
Skills	`skill_id PK, skill_name`	Skill catalogue.
Languages	`language_id PK, language_name`	Language catalogue.
User_Skills	`user_id FK, skill_id FK (composite PK)`	Student–skill mapping.
User_Languages	`user_id FK, language_id FK, level (composite PK)`	Student–language mapping with proficiency level.

User_Experiences	`id PK, user_id FK, company_name, position, description, start_date, end_date`	Work experience entries.
User_Educations	`id PK, user_id FK, university, department, gpa, start_date, end_date`	Education history entries.

Relationships

- One user can have one Student_Profile or one Professor_Profile based on role.
- One professor can create many Jobs.
- One student can apply to many Jobs via Job_Applicants; UNIQUE(job_id, user_id) prevents duplicate applications.
- Users have many User_Skills, User_Languages, User_Experiences, and User_Educations.

The full Entity Relationship Diagram is provided in the GitHub repository, showing all 12 tables with column types and FK relationships.

4. Proposed SQL Features

The proposed database system will use several SQL features to ensure data integrity, consistency, performance, and maintainability. These features support both the database structure and the assistant recruitment workflow.

- **Primary Keys** Used to uniquely identify each record (e.g., user_id, job_id).
- **Foreign Keys** Used to connect related tables and maintain referential integrity (e.g., Users → Profiles, Jobs → Applications).
- **Check Constraints** Used to enforce valid values (e.g., role = student/professor, status = pending/approved/rejected).
- **Unique Constraints** Used to prevent duplicate data (e.g., unique emails, one application per job per user).
- **Identity Columns** Used for automatic ID generation in main tables.
- **Default Values** Used for fields like created_at, is_active, and default application status.
- **Indexes** Used to improve query performance on frequently accessed tables.
- **Filtered Indexes** Used to optimize queries on active records (e.g., only active job postings).
- **Views** Used to simplify complex queries (e.g., active jobs, applicant lists, user applications).
- **Common Table Expressions (CTEs)** Used to write clearer and more structured queries, especially when combining data from multiple tables. For example, CTEs can be used to generate a complete CV by merging user information, education, experience, skills, and languages into a single result set.

Relationships

- One-to-One: Users ↔ Profiles
- One-to-Many: Users → Jobs, Users → Applications
- Many-to-Many: Users ↔ Skills / Languages
- **Appropriate Data Types** Data types such as INT, NVARCHAR, DATE, DATETIME2, DECIMAL, and BIT are used for efficient and accurate storage.

Summary

These SQL features ensure that the system is structured, efficient, and scalable. They support reliable user management, organized job postings, and a clear application process within the university assistant recruitment system.

5. Technology Stack

Layer	Technology	Role
Frontend	Flutter	Cross-platform mobile app for Android and iOS.
Backend	Node.js (Express.js)	RESTful API — business logic, authentication, data processing.
Database	Azure SQL	Relational database storing users, postings, applications, and profiles.
Auth	JWT	Secure authentication and role-based authorization.
API Comm.	REST API (JSON/HTTP)	Communication between frontend and backend.
Version Ctrl.	Git & GitHub	Source code management and team collaboration.